

# Exploiting Insecure Output Handling in LLM Applications

---

Author: Moe Htet Ar Kar (Phoe Cho)

## 1. Vulnerability Concept & Mechanics

Insecure output handling occurs when an LLM application blindly trusts and renders model-generated output directly into downstream components, such as web interfaces, command interpreters, or database-related workflows, without appropriate sanitization, validation, or context-aware encoding.

Because LLM outputs can be influenced by malicious input or prompt injection, insecure output handling can create an indirect pathway to traditional application vulnerabilities.

## 2. Attack Vector & Exploitation Methodology

### Payload Injection

Assess whether an attacker can influence the model to generate output containing potentially dangerous HTML, JavaScript, SQL syntax, or operating-system commands.

### Unsanitized Rendering

Determine whether the model response is passed directly into a web frontend, HTML sink, command processor, or other security-sensitive component without appropriate validation or encoding.

### Triggering Downstream Execution

Verify whether the generated output is interpreted as executable content rather than treated as untrusted data.

## 3. Validation & Impact

- **Impact:** Potential Cross-Site Scripting (XSS), command execution, SQL injection, session compromise, or other downstream application vulnerabilities.
- **Validation:** Confirm whether maliciously influenced model output is interpreted or executed by the downstream component.

## 4. Remediation & Hardening

- Apply rigorous context-aware output encoding before rendering LLM responses.
- Validate and sanitize model output according to the destination context.
- Implement strict Content Security Policy (CSP) headers to reduce the impact of potential XSS vulnerabilities.
- Treat LLM-generated output with the same security rigor applied to other untrusted input.